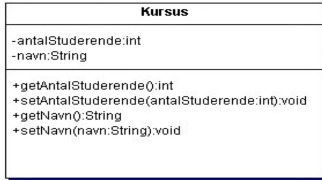


Relación 2

Ejercicio 1. UML es un lenguaje "universal". Escribe el código Java y Ruby correspondiente a la declaración de la siguiente clase en danés (incluyendo la declaración de atributos y la cabecera de los métodos).



Ruby:

```
class Kursus
  attr_accessor :antalStuderende, :navn
end
```

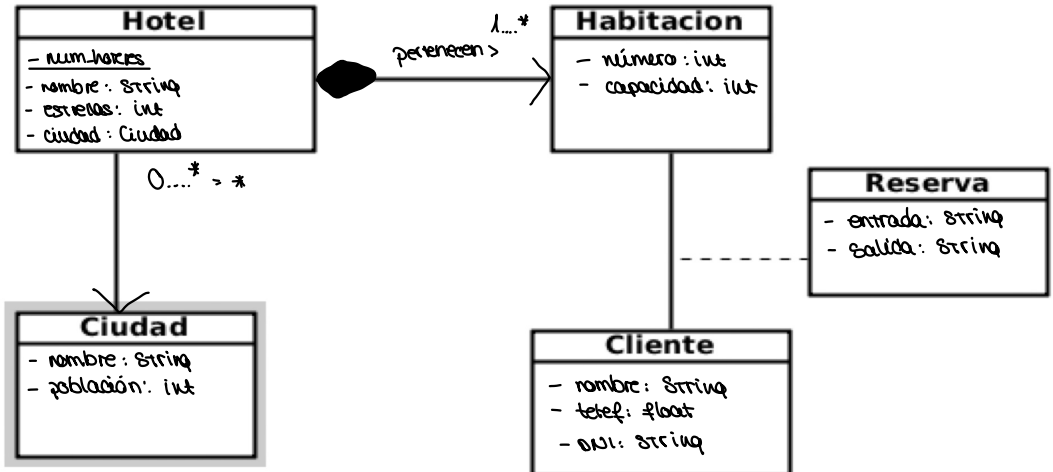
no se puede usar en atrib. de clase

Java:

```
public class Kursus {
    private int antalStuderende;
    private String navn;

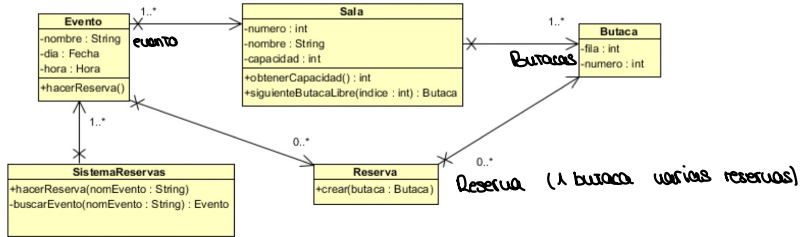
    public int getAntal();
    public void setAntal(int antal);
    public String getNavn();
    public void setNavn(String navn);
}
```

Ejercicio 2. El siguiente diagrama de clases representa hoteles con sus habitaciones y las reservas hechas por sus clientes:



- Complétalo con los atributos de las clases y el nombre, roles, multiplicidad y navegabilidad de las asociaciones, haciendo las suposiciones que consideres oportunas.
- Implementa en Java el resultado obtenido.
- Implementa en Ruby el resultado obtenido.

Ejercicio 3. A partir del siguiente diagrama de clases.



- Indica el nombre o roles de las asociaciones.
- ¿La asociación que existe entre Sala y Butaca podría ser una agregación? ¿Y una composición?
- Partiendo de una sala, ¿podría saberse para qué eventos está siendo usada?
- Escribe el código Java correspondiente.
- Escribe el código Ruby correspondiente.

a) Evento - Sala :

No hay rol en el diagrama → asumimos que es el nombre de la clase en minúscula (por defecto)

En singular para un solo obj. y en plural para una colección

No hay navegabilidad → no atributo de eventos en sala, pero entonces ¿Porque esa cardinalidad?

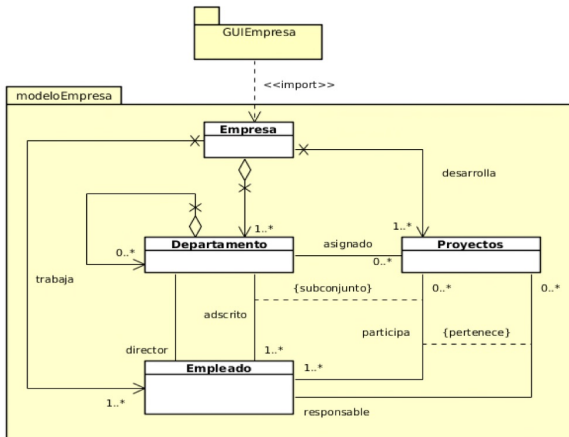
↳ En una sala estamos informando de que se pueden representar varios eventos. No es común encontrar eso (info. extra)

b) Agregación si. Una sala tiene varias butacas.

una composición también. Una butaca solo puede estar en una sala y una sala en esencia es un conjunto de butacas.

c) No, porque no hay navegabilidad

Ejercicio 4. A partir del siguiente diagrama de clases, responde a las cuestiones plateadas:



- A) La asociación que hay entre Empresa y Departamento, ¿es de tipo agregación o composición? ¿podría ser del otro tipo? Razona por qué.

Agregación. Podría ser composición porque un depto. no puede estar en varias empresas a la vez, ni puede no pertenecer a ninguna, carecería de sentido su existencia.

- B) ¿Cuál es la multiplicidad de dicha asociación en el lado de Empresa? ¿podría ser distinta? Razona por qué.

La multiplicidad es 1, no podría ser diferente pues un depto. no puede pertenecer a más de una empresa, ni a menos.

- C) La asociación que hay entre Departamentos, ¿es de tipo agregación o composición? ¿podría ser del otro tipo? Razona por qué

Agregación. No puede ser composición porque un depto. al tiene sentido en si mismo, sin pertenecer a ningún otro.

- D) ¿Qué son GUIEmpresa y modeloEmpresa?

Paquetes.

- E) La relación que hay entre GUIEmpresa y Empresa ¿de qué tipo es?, es decir ¿qué significa <<import>> sobre ella?

Relación de dependencia. Import significa que GUIEmpresa requiere de Empresa para poder funcionar. utilizará alguna instancia de Empresa.

- F) ¿Cuál es la navegabilidad de la asociación adscrito entre Empleado y Departamento? ¿y de la asociación desarrolla entre Empresa y Proyectos?

Navegabilidad bidireccional.

La navegabilidad va solo en sentido de proyectos, es decir, la empresa conoce info. acerca de los proyectos pero no al revés.

- G) ¿Qué representa la línea discontinua entre las asociaciones adscrito y participa y cuál es su significado en este diagrama?

Es una especificación de las asociaciones.

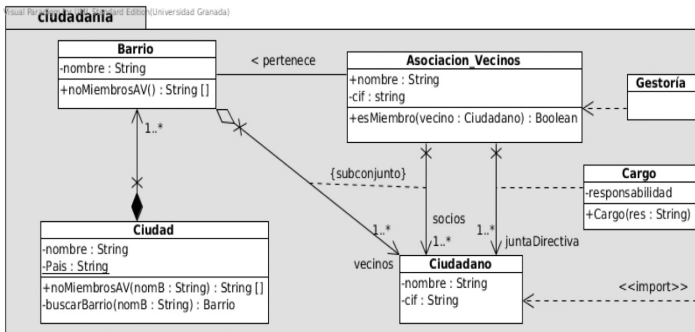
los empleados que participan en un proyecto deben ser un subconjunto de empleados adscritos a un depto.

- H) Hay dos asociaciones entre Proyectos y Empleado, ¿qué representan?

- I) ¿Por qué hay 0..* y no 1..* en la multiplicidad de la asociación entre Empleado y Proyectos?

Porque para que un proyecto exista debe pertenecer mínimo a 1 empleado porque si no pertenece a nadie, no se desarrolla dicho proyecto.

Ejercicio 5 (Examen 13/14). Partiendo del siguiente diagrama de clases de UML, resolver las cuestiones planteadas a continuación.



Ciudadano.rb → archivo

module Ciudadania

class Ciudadano

end

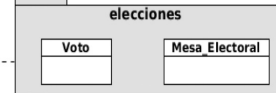
no podemos importar un módulo completo hay que importar aquello a donde se le en la con require-relat

libro.rb require_relative 'Ciudadano.rb'

module elecciones

class libro

variable = Ciudadania::Ciudadano.new

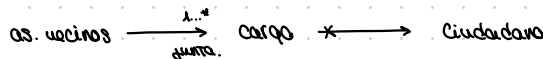


- A) ¿Qué cambio habría que hacer en el diagrama de clases para modelar que un barrio puede tener varias asociaciones de vecinos?

hay que cambiar la cardinalidad de asociación de vecinos a 1..*

- B) ¿Desde un objeto de la clase Ciudadano puede saberse si es presidente de la asociación de vecinos de su barrio? ¿Por qué?

no, porque no hay navegabilidad. Quedaría así:



- C) ¿De qué tipo es la relación entre Ciudad y Barrio? ¿qué significa?

Composición. Una ciudad en esencia está formada por barrios. Un barrio no tiene sentido sin pertenecer a una ciudad y solo puede pertenecer a 1.

- D) ¿De qué tipo es la relación entre Barrio y Ciudadano? ¿qué significa? ¿podría ser de otro tipo?

Agregación. un ciudadano pertenece a un barrio, pero podría pertenecer a más. y un barrio no es en su esencia un objeto de ciudadanos.
no podría ser composición

- E) ¿Qué significa la línea discontinua con la indicación "{subconjunto}" entre las asociaciones de Barrio---Ciudadano y Asociacion_Vecinos---Ciudadano?

Especificación de las asociaciones.

un subconjunto de los ciudadanos del barrio pertenecen...

los socios de la asociación deben ser un subconjunto de los ciudadanos del barrio

- F) Si la junta directiva de una asociación de vecinos está formada por exactamente 6 ciudadanos, ¿cómo lo indicarías en el diagrama de clases?

Cambia el 1..* por 6.

- G) ¿Qué significa que la clase Cargo esté ligada a la asociación entre Asociacion_Vecinos y Ciudadano?

Es info. asociada a la asociación.

- H) Implementa en Ruby los consultores/modificadores básicos de la clase Asociación_Vecinos.

```
def nombre
  @nombre
end
```

```
def nombre = nom
  @nombre = nom
end
```

```
def cif
  @cif
end
```

```
def cif = cif_nuevo
  @cif = cif_nuevo
end
```

- I) ¿Cómo debería ser el constructor de Ciudadano para que inicialice el estado completo de una instancia? Implementalo en Java y Ruby.

```
def initialize(nom, cif)
  @nombre = nom
  @cif = cif
end
```

```
public Ciudadano(String n, String c){
    this.nombre = n;
    this.cif = c;
} // se pone siempre
```

- J) Suponiendo que la visibilidad de todas las clases del diagrama es pública y que estamos codificando la clase Voto en Java ¿A qué otras clases puede acceder ésta usando directamente su nombre, sin indicar el nombre del paquete al que pertenecen?

Voto accede a mesa-electoral y a la pública de ciudadano.

- K) Define en Java los atributos de referencia de la clase Barrio.

```
private ArrayList<Ciudadano> vecinos;
private Asociacion_Vecinos asoc_vecinos;
```

(no hay atrib. de ciudad porque es la parte, no el todo)
(de la composición y no hay navegabilidad.
si no por nada debe ser el mismo nombre de la clase pero minúsculas.

L) ¿Desde qué clases es accesible el atributo nombre de la clase Asociacion_Vecinos?

Solo desde las clases de dentro del paquete.
Desde elecciones no, porque solo importa ciudadanía.

M) Implementa en Java y Ruby las clases Asociacion_Vecinos y Mesa_Electoral completas
(Ojo: no incluir nada que no esté en el diagrama de clases proporcionado).

```
module ciudadanía
  class AsociacionVecinos
    attr_accessor :nombre
    def initialize (cif, b)
      @cif = cif
      @socios[]
      @juntaDirectiva[]
      @barrio = b
    end

    def esmiembro (vecino)
      //implementación (devuelve un bool)
    end
  end
end
```

```
require_relative 'asociaciondevecinos.rb'
module elecciones
  class MesaElectoral
  end
end
```

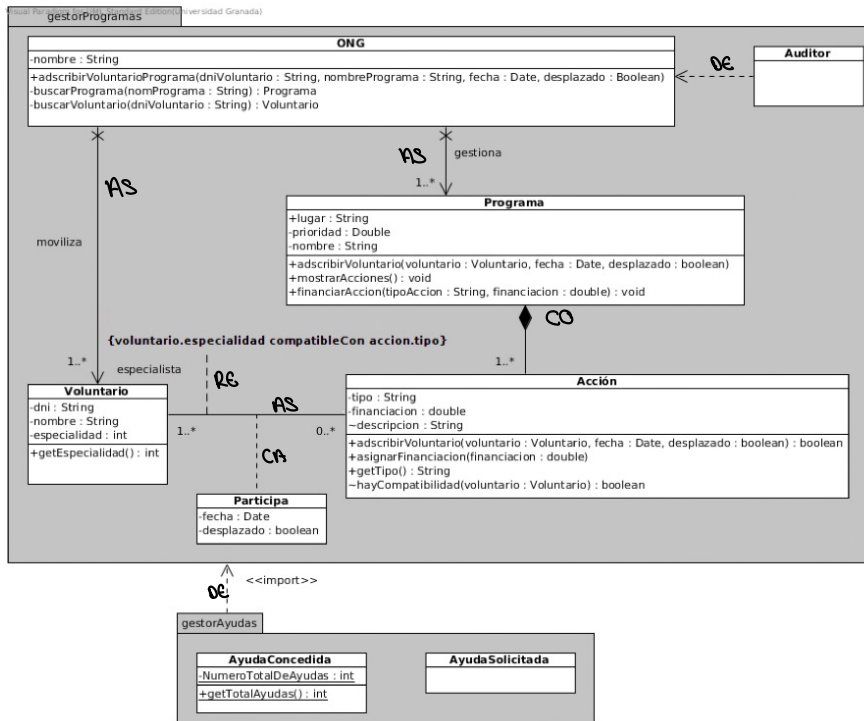
```
package ciudadanía
import java.util.ArrayList
public class AsVec {
    public String nombre;
    private String cif;
    private ArrayList<ciudadano> socios;
    private " " " juntaDirectiva;
    private Barrio b;

    public bool esmiembro (Ciudadano vecino){
        //implem.
    }
}
```

```
package elecciones
import ciudadanía.Ciudadano;
public class mesaElectoral{
}
```

para usar algo del otro paquete se hace igual
que si la clase estuviera dentro del mismo paquete.

Ejercicio 6 (Examen 12/13). Partiendo del siguiente diagrama de clases de UML, responde verdadero (V) o falso (F) a las cuestiones:



Desde la clase AyudaSolicitada se puede acceder a todos los elementos públicos del paquete GestorProgramas.	V	(porque hay un import)
Un voluntario puede participar en cualquier acción de un programa sin ningún tipo de restricción.	F	
El estado de un objeto de la clase Auditor viene determinado por el estado de un objeto de la clase ONG.		
Un voluntario podría participar en acciones de distintos programas.	V	
Un voluntario puede pertenecer a varias ONG.	F	solo 1
Cuando se define un objeto de la clase Acción, éste tiene que asociarse a un determinado objeto de la clase Programa.	V	
En una acción puede participar más de un voluntario como especialista.	V	
Desde un objeto de la clase ONG se puede llegar a conocer a todos los especialistas de una determinada acción en un programa.	V	
El estado de un objeto Voluntario está exclusivamente determinado por su dni, nombre y especialidad.	F	Debe tener un array de acción
Todos los métodos de la clase Acción pueden ser accedidos desde la clases AyudaConcedida.	F	visibilidad de paquete (v) no.

2

Ejercicio 7 (Examen 13/14). Partiendo del diagrama de clases del **ejercicio 6** responde a las siguientes preguntas:

- A) Usando la siguiente nomenclatura: AS = Asociación, CO = Composición, AG = Agregación, DE = Dependencia, CA = Clase Asociación y RE = Restricción, etiqueta los elementos correspondientes en el propio diagrama de clases. *hacha arriba*
- B) Implementa en Java y Ruby las clases Accion y AyudaConcedida.

```
module gestorProgramas
  class Accion
    attr_accessor :descripcion
    def initialize (t, f, d)
      @tipo = t
      @financiacion = f
      @participa = []
    end
  end
end
```

para hacerlo publico porque es ~

(todas las metodos publicas)

```
require_relative 'gestorProgramas'
module gestorAyudas
  class AyudaConcedida
    @@numTotalAyudas
    def self.get TotalAyudas
      @@numTotalAyudas
    end
  end
end
```

Como lo hago para un modulo completo.

```
Package GestorProgramas
public class Accion {
    private String tipo;
    private double financiación;
    String descripcion;
    private ArrayList <Participa> participa;
    (añaden los metodos)
}
```

```
Package GestorAyudas
public class AyudaConcedida
    private static int numTotalAyudas;
    private static int getTotalAyudas();
    return numTotalAyudas;
}
```

Ejercicio 8. Partiendo del diagrama de clases del **ejercicio 6**, implementa en Java y en Ruby los atributos de referencia de todas las clases.

Accion y AyudaConcedida → Ej. 7.

```
module GestorProgramas
  class ONG
    def initialize
      @voluntarios = []
      @programas = []
    end
  end
```

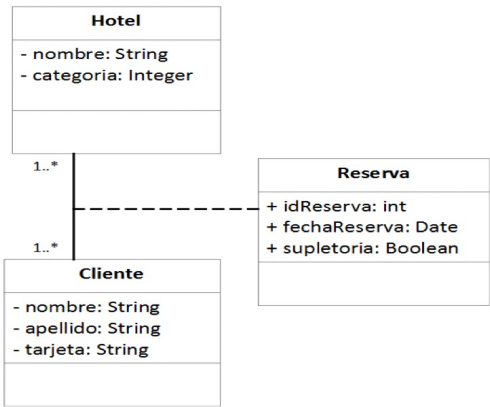
```
class Programa
  def initialize
    @acciones = []
  end
end
```

```
class voluntario → Accion igual
  def initialize()
    @participa = []
  end
end
class Participa
  def initialize (v, a)
    @voluntario = v
    @accion = a
  end
end
```

Auditor y AyudaSolicitada no tienen.

En java igual pero declarados como: private ArrayList < > ... ;

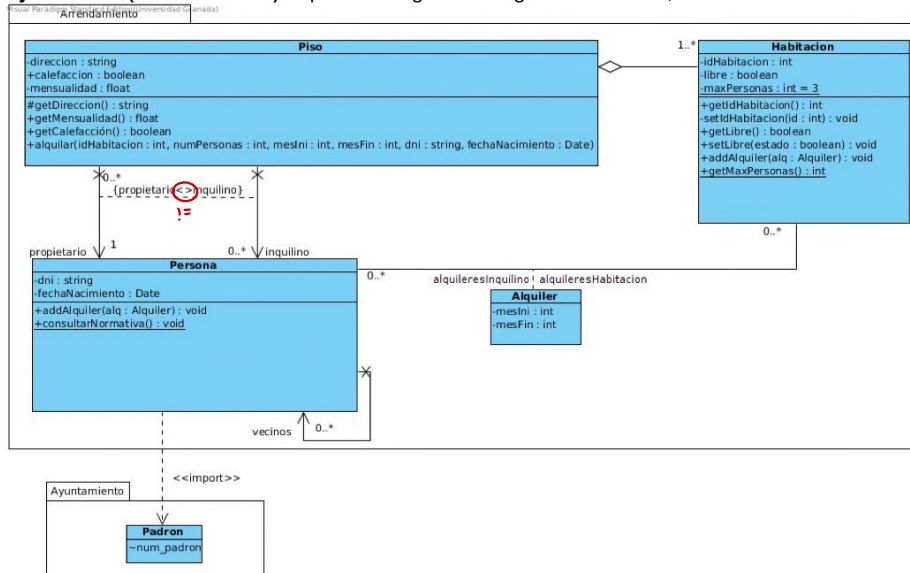
Ejercicio 9 (Examen 12/13). Cuando se implemente en Java el siguiente diagrama de clases, responde si las afirmaciones son verdaderas (V) o falsas (F)



Hotel tendrá dos atributos de referencia, uno relativo a las reservas y otro relativo a los clientes	f
Reserva tendrá los atributos de referencia: private Hotel hotel; private Cliente cliente;	f

solo relativo a reserva.
son arrays.

Ejercicio 11 (Examen 15/16). A partir del siguiente diagrama de clases,



Responde verdadero (V) o falso (F) a las siguientes cuestiones:

La asociación <i>Piso-Habitación</i> es de agregación	V
<i>{propietario <-> inquilino}</i> es una restricción	F
Una persona puede saber de qué pisos es propietaria	F
<i>Alquiler</i> es una clase asociación	V
<i>vecinos</i> es un rol	V
Puede haber una habitación sin inquilinos	V
<i>Piso</i> tiene un atributo de referencia de la clase <i>Persona</i>	V

El método <i>getDireccion</i> de <i>Piso</i> es método de clase, no de instancia	F
Desde la clase <i>Piso</i> se puede acceder directamente al atributo <i>dni</i> de un inquilino	F
Desde la clase <i>Persona</i> se puede acceder directamente al atributo <i>num_padron</i> de un objeto de la clase <i>Padron</i>	
Los siguientes objetos de <i>Piso</i> tienen la misma identidad: <i>Piso piso1= new Piso("Calle Alta", true, 600)</i> <i>Piso piso2= new Piso("Calle Alta", true, 600)</i>	F

✓
no hay navegabilidad

instancia de no subobjeto.
esta pregunta es #
es pu.

Tienen mismo estado

Ejercicio 12. Escribe el código necesario para hacer que *piso1* y *piso2* sean idénticos.

Piso piso2 = piso1;

Ejercicio 13. ¿Desde la clase *Padrón* se puede acceder al método *getCalefacción* de *Piso*? Justifica tu respuesta.

no, está en otro paquete

Ejercicio 14. Indica si es correcto implementar en Ruby para la clase Padrón lo siguiente:

- a) `attr_reader :dirección, :mensualidad` } Estaría bien para
 b) `attr_accessor :calefacción` } qto

Justifica tu respuesta.

Para padrón no porque no tiene dichos atributos.

Ejercicio 15. Teniendo en cuenta sólo la información presentada en el diagrama de clases del ejercicio 11, implementa en Java y Ruby las clases *Persona*, *Alquiler* y *Padrón*. Añade sus constructores para inicializar los atributos y métodos consultores para todos los atributos. No te olvides de incluir también el código necesario para indicar que cada clase está dentro de un paquete o módulo, y si precisa algo de otro paquete o módulo. Supón que en Ruby cada clase se implementa dentro de un fichero con el mismo nombre que la clase y que están todas en la misma carpeta. Dado que no tenemos información sobre los métodos de *Persona*, implementa solo sus cabeceras.

¿Cuál necesita al otro. Es persona quien importa? o todo el paquete amend. como se usa ----> caso que es que padrón usa inst. de persona.

Package Amendamiento
 (import Ayuntamiento Padrón) ✓

```
public class Persona {
    private String dni;
    private Date fechaNacimiento;
    private Habitación hab;
    private ArrayList<Persona> vecinos; // como b
    // atributo de alquiler? recordo
```

```
public Persona (String dni, Date fn, Habitación h) {
    this.dni = dni;
    this.fechaN = fn;
    vecinos = new ArrayList<>();
    this.hab = h;
}
```

```
public static void consultarNormativa() {}
```

```
public void addAlquiler(Alquiler aq) {}
```

} // tanto consultores.

```
public class Alquiler {
    private int mesIni;
    private int mesFin;
```

```
public Alquiler (int ini, int fin) // tanto consultores.
```

```
    this. ...
```

```
}
```

```
}
```

Package Alquileramiento

import Alquileramiento.Persona;

public class Padron {

int num-padron

public Padron (int padron) {

this.num-padron = padron;

}

}

→ ¿lo importa el paquete o la clase Padron?

→ aunque vaya import por la dependencia no hay atributo persona por no existen los atributos de dependencia.

public int getnum-padron() {

return num-padron;

}

module Ayto

~~include alquileramiento~~

class Padron

def initialize (np)

@num-padron = np

end

def getnum-padron

@num-padron

end

end

module Alquileramiento

class Persona

def initialize (dni, fn

@dni = dni

@fecha_nacimiento = fn

@habitacion = holo

@vecinas = [] → ¿como recorrer

alquiler?

end

def self.consultasNormativas ()

end

farran constructores

ayto::padron.new

→ se deja así?

Pers. alquiler packon const. consultor

```
Package Ayuntamiento;  
import java.util.ArrayList;  
import Ayuntamiento.Packon;
```

```
class Persona {  
    private String dni;  
    private Date fechaNac;  
    private ArrayList<Alquiler> alquileres;  
    private ArrayList<Persona> vecinos;
```

```
    public Persona(String dni, Date f) {  
        this.dni = dni;  
        this.fechaNac = f;  
        alquileres = new ArrayList<>();  
        vecinos = new ArrayList<>();  
    }  
>
```

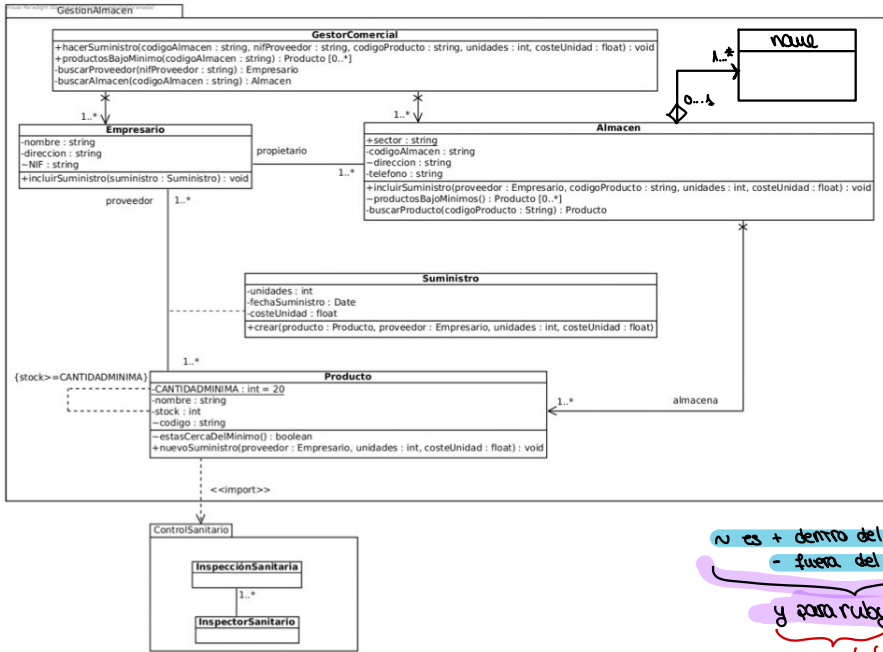
```
    public void addAlquiler(Alquiler a) {}  
    public static void consultarNormativa() {};
```

```
// Consultores públicos.
```

>

Require-Relative 'Packon.rb'
Persona.rb

Ejercicio 16 (Examen 16/17). Dado el siguiente diagrama de clases:



~ es + dentro del paquete.
- fuera del paquete.

y para ruby

publico?!

- 1) Dibuja cómo representarías en UML la relación entre la clase **Almacen** y una nueva clase **Nave**, sabiendo que un almacén está compuesto por varias naves. *Partición: pa una nave pueda existir sin un almacen*

- 2) Responde verdadero (V) o falso (F).

Si la clase InspeccionSanitaria tuviese atributos privados, éstos serían accesibles desde la clase Producto puesto que hemos hecho el import del paquete.	F
El stock de un Producto puede ser de menos de 20 unidades.	F
Un mismo Producto lo pueden suministrar varios Empresarios .	V
El atributo sector de la clase Almacen es accesible desde los métodos de la clase Producto .	V

pa hay restricción
pa 1..*
pa público.

- 3) Teniendo en cuenta sólo la información presentada en el diagrama de clases, implementa en Java la clase **Producto** y en Ruby la clase **Almacen**. Añade e implementa sus constructores para inicializar los atributos. Cuando estos atributos sean colecciones, inicialízalos con la lista vacía. Incluye los métodos consultores (no los modificadores) para todos los atributos.

Package GestionAlmacen;
import java.util.ArrayList;
public class Producto {
 private static final int CANTIDADMINIMA = 20;
 private String nombre;
 private int stock;
 String codigo;
 private Empresario proveedor; *→ Debe ser array?*

private
ml, :mz
→ mas errores

```

public Producto (String nom, int stock, String cod, Empresa e){
    this.nombre = nom;
    this.stock = stock;
    this.codig = cod;
    this.proveedor = e;
}

bool estasCercaDelMinimo(){};

public void nuevoSuministro (...){};

// Consultas.
}

```

module GestionAlmacen

class Almacen

@@begin → hay q inicializarlo? fuera del constructor? Como se hace publico? con access?

def initialize(ca, dir, tel, prop)

@codigoAlmacen = ca

@direccion = dir

@telefono = tel

@propietario = prop → se llama asi?

@productos = []

end

private → Gmo indicar que solo para este método

def buscarProductos (codPro){}

end

def productosBajominimos(){};

end

def incluirSuministro(...){};

end

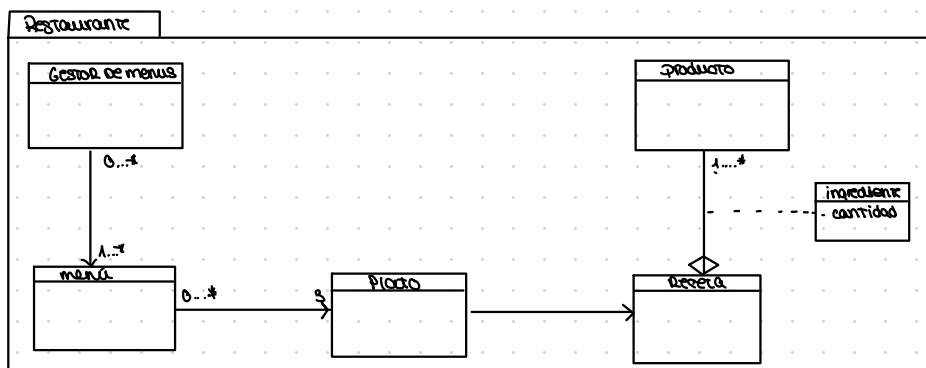
Constructores.

end

end necesario para acabar modulo? → Si

Ejercicio 10. Modela (obtén el diagrama de clases) de los siguientes supuestos, incluyendo en el modelo todo lo que conozcas del tema:

- En un restaurante se dispone de un Gestor de Menú encargado de elaborar los menús para cada mes, cada menú está compuesto por tres platos: un primero, un segundo y un postre. A cada plato le corresponde una receta. Los ingredientes de la receta lo componen una lista de productos y las cantidades necesarias de cada producto. Para elaborar un menú para un día concreto hay que proporcionar un primero, un segundo y un postre.
- En una carrera de relevos se inscriben equipos de atletas, cada equipo lo forman 4 deportistas, cada uno corre una distancia determinada tras la cual entrega el testigo al siguiente corredor del equipo. Para participar hay que inscribirse en la carrera. Una vez finalizada, reciben medalla (oro, plata y bronce) los tres equipos que primero lleguen a la meta, también se proporciona medalla (oro, plata y bronce) a los corredores que hayan obtenido los tres mejores tiempos.
- Un alumno está matriculado en una serie de asignaturas de una determinada titulación, las asignaturas están formadas por grupos, de tal forma que cada alumno asiste a clase en un determinado grupo. Al comienzo del curso se le asigna a cada alumno un grupo de cada asignatura en la que esté matriculado.



1. 11

teresa. usar .length o .size en vez de un final

Gj 3

```
public class Evento {  
    private String nombre;  
    private fecha dia;  
    private hora hora;  
    private Sala sala;  
    private ArrayList<Reserva> reservas;  
    public Evento(...) {};
```

```
    public void hacerReserva() {};
```

}

```
public class Sala {  
    private int numero;  
    private String nombre;  
    private int cap;  
    private ArrayList<Reserva> reservas;
```

```
    public Sala(...) {};
```

}

```
public class Reserva {  
    private Reserva but;  
  
    public Reserva (Reserva b) {};  
    public void crear (Reserva reserva) {};
```

}

```
public class SL {  
    private ArrayList<Evento> eventos;  
  
    public SL(...) {};  
    private Evento buscarEvento(...) {};  
    public void hacerReserva (...) {};
```

}

```
class Evento  
def initialize (n, d, h, s)  
    @nombre = n;  
    @dia = d;  
    @hora = h;  
    @sala = s;    @reservas = [];  
end
```

```
def hacerReserva()  
    (...)
```

```
end
```

```
end
```

```
class Sala  
def initialize (n, nom, cap)  
    @numero = n  
    @nombre = nom  
    @capacidad = cap  
    @reservas = []  
end
```

```
end
```

